- CPSC 375: High-Performance Computing

CPSC 375: High-Performance Computing

Spring 2026

- Syllabus
- Schedule
- Upload
- The Cluster Project

More on OpenMP Programming

In this lab you will explore more advanced OpenMP constructs used to coordinate threads, control execution, and improve performance. You will learn how to synchronize threads, control loop scheduling, use parallel sections and tasks, and measure parallel performance.

Exercise 1: Using Barrier

A barrier forces all threads to wait until every thread reaches the same point.

```
#include <stdio.h>
#include <omp.h>

int main() {
   #pragma omp parallel
   {
      printf("Thread %d before barrier\n", omp_get_thread_num());

      #pragma omp barrier

      printf("Thread %d after barrier\n", omp_get_thread_num());
   }
}
```

Compile and run

```
$ gcc -fopenmp -o barrier barrier.c
$ ./barrier
```

Questions

- Do all threads print before barrier first?
- Why might barriers be necessary in parallel programs?

Exercise 2: Using Master

The `master` directive allows only the “master” thread to execute a block.

```
#include <stdio.h>
#include <omp.h>

int main() {

#pragma omp parallel
{
```

```
    printf("Thread %d working\n", omp_get_thread_num());

    #pragma omp master
    {
        printf("Master thread %d doing special work\n", omp_get_thread_num());
    }
}

}
```

Compile and run

```
$ gcc -fopenmp -o master master.c
$ ./master
```

Questions

- Which thread executes the master block?
- How many times is the message printed?

Exercise 3: Using Single

The `single` directive allows one thread (any thread) to execute a block.

```
#include <stdio.h>
#include <omp.h>

int main() {

#pragma omp parallel
{
    printf("Thread %d running\n", omp_get_thread_num());

    #pragma omp single
    {
        printf("One thread executes this section\n");
    }
}

}
```

Compile and run

```
$ gcc -fopenmp -o single single.c
$ ./single
```

Questions

- Which thread executes the single block?
- Does it always use thread 0?

Exercise 4: Using Atomic

The `atomic` directive protects a single memory update operation.

```
#include <stdio.h>
#include <omp.h>

int main() {
   int counter = 0;
```

```
   #pragma omp parallel for
   for(int i = 0; i < 1000; i++)
   {
      #pragma omp atomic
      counter++;
   }
   printf("Counter = %d\n", counter);
}
```

Compile and run

```
$ gcc -fopenmp -o atomic atomic.c
$ ./atomic
```

Questions

- Why is `atomic` usually faster than `critical`?
- When might `atomic` not be sufficient?

Exercise 5: Removing Implicit Barriers with nowait

OpenMP loops normally end with a barrier. The `nowait` clause removes this synchronization.

```
#include <stdio.h>
#include <omp.h>

int main() {

   #pragma omp parallel
   {
      #pragma omp for nowait
      for(int i = 0; i < 8; i++)
      {
         printf("Thread %d loop iteration %d\n", omp_get_thread_num(), i);
      }
      printf("Thread %d continues immediately\n", omp_get_thread_num());
   }
}
```

Compile and run

```
$ gcc -fopenmp -o nowait nowait.c
$ ./nowait
```

Questions

- Do some threads continue earlier?
- Why might removing barriers improve performance?

Exercise 6: Loop Scheduling

OpenMP distributes loop iterations among threads using scheduling policies.

```
#include <stdio.h>
#include <omp.h>

int main() {

   #pragma omp parallel for schedule(static)
```

```
    for(int i = 0; i < 16; i++)
    {
        printf("Thread %d executes iteration %d\n", omp_get_thread_num(), i);
    }
}
```

Compile and run

```
$ gcc -fopenmp -o schedule schedule.c
$ ./schedule
```

Experiment

Change:

```
schedule(static)
```

to:

```
schedule(dynamic)
```

Questions

- How does iteration assignment change?
- When would dynamic scheduling be useful?

Exercise 7: Controlling Chunk Size

Modify the loop (you may need to increase the number of iterations):

```
#pragma omp parallel for schedule(static,2)
```

Compile and run

```
$ gcc -fopenmp -o chunk chunk.c
$ ./chunk
```

Questions

- How are iterations assigned now?
- What happens if the chunk size becomes 4?

Exercise 8: Parallel Sections

Sections allow different blocks of code to run concurrently.

```
#include <stdio.h>
#include <omp.h>

int main() {

#pragma omp parallel sections
{
   #pragma omp section
   printf("Section A executed by thread %d\n", omp_get_thread_num());

   #pragma omp section
   printf("Section B executed by thread %d\n", omp_get_thread_num());
```

```
    #pragma omp section
    printf("Section C executed by thread %d\n", omp_get_thread_num());
    }
}
```

Compile and run

```
$ gcc -fopenmp -o section section.c
$ ./section
```

Questions

- Do sections execute in parallel?
- Does the output order change each run?

Exercise 9: Task Parallelism

Tasks allow threads to execute independent units of work dynamically.

```
#include <stdio.h>
#include <omp.h>

void work(int id)
{
    printf("Task %d executed by thread %d\n", id, omp_get_thread_num());
}

int main() {
   #pragma omp parallel
   {
      #pragma omp single
      for(int i = 0; i < 8; i++)
      {
         #pragma omp task
         work(i);
      }
   }
}
```

Compile and run

```
$ gcc -fopenmp -o task task.c
$ ./task
```

Questions

- Which threads execute the tasks?
- Do tasks execute in order?
- Why is `single` required?

Exercise 10: Measuring Performance

OpenMP provides timing functions to measure execution time.

```
#include <stdio.h>
#include <omp.h>

int main() {
   double start = omp_get_wtime();
   long sum = 0;
```

```
    #pragma omp parallel for reduction(+:sum)
    for (long i = 0; i < 100000000; i++)
    {
        sum += i;
    }
    double end = omp_get_wtime();

    printf("Sum = %ld\n", sum);
    printf("Time = %f seconds\n", end - start);
}
```

Compile and run

```
$ gcc -fopenmp -o timing timing.c
$ export OMP_NUM_THREADS=1
$ ./timing
$ export OMP_NUM_THREADS=4
$ ./timing
$ export OMP_NUM_THREADS=8
$ ./timing
```

Questions

- Does runtime improve with more threads?
- Why might performance stop improving?

Exercise

Write a program that:

- Creates an array of 10,000 numbers
- Uses `parallel for` with dynamic scheduling
- Uses `reduction` to compute the sum
- Measures execution time using `omp_get_wtime()`

- **Welcome: Sean**
  - LogOut

Trinity College
HARTFORD CONNECTICUT